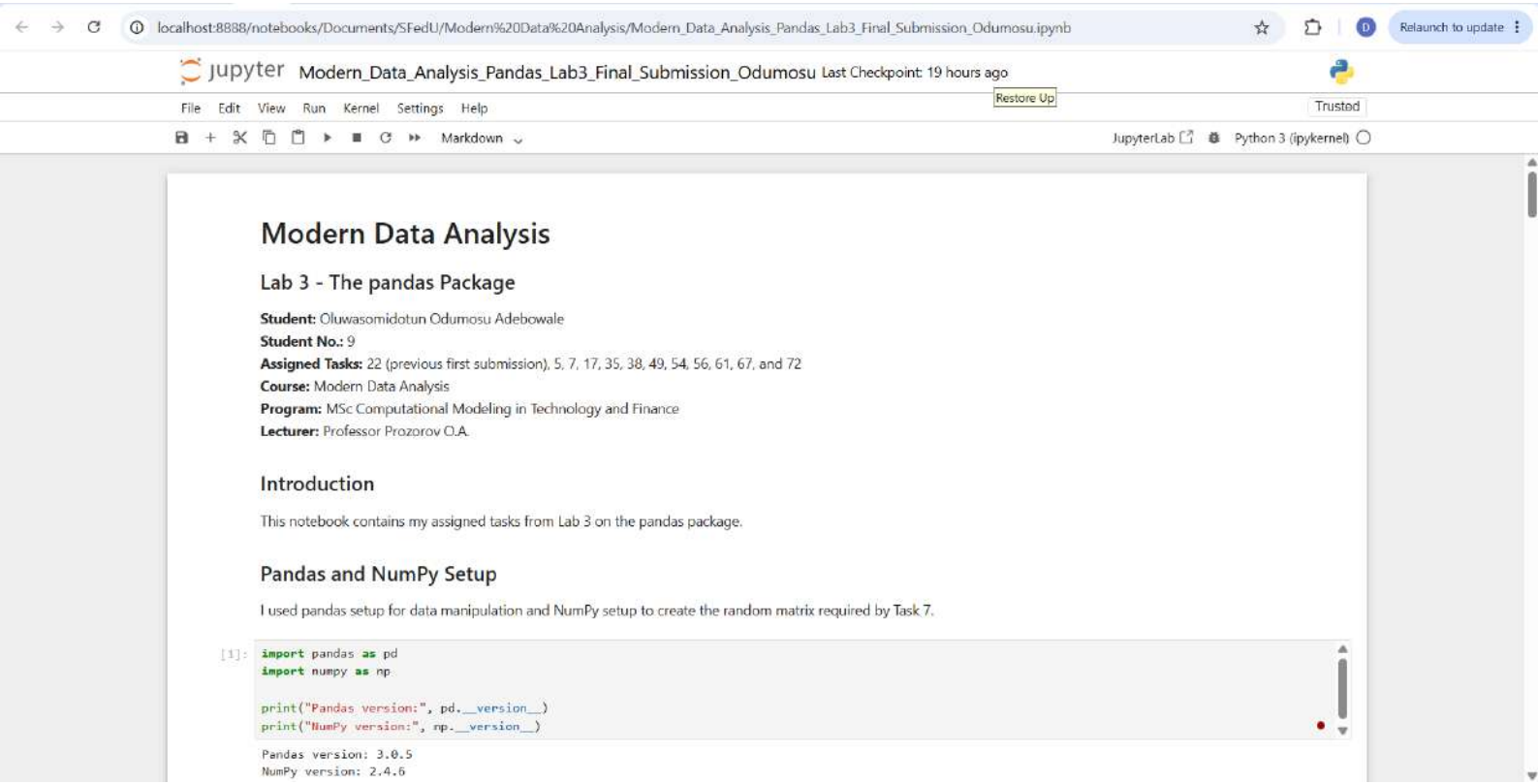
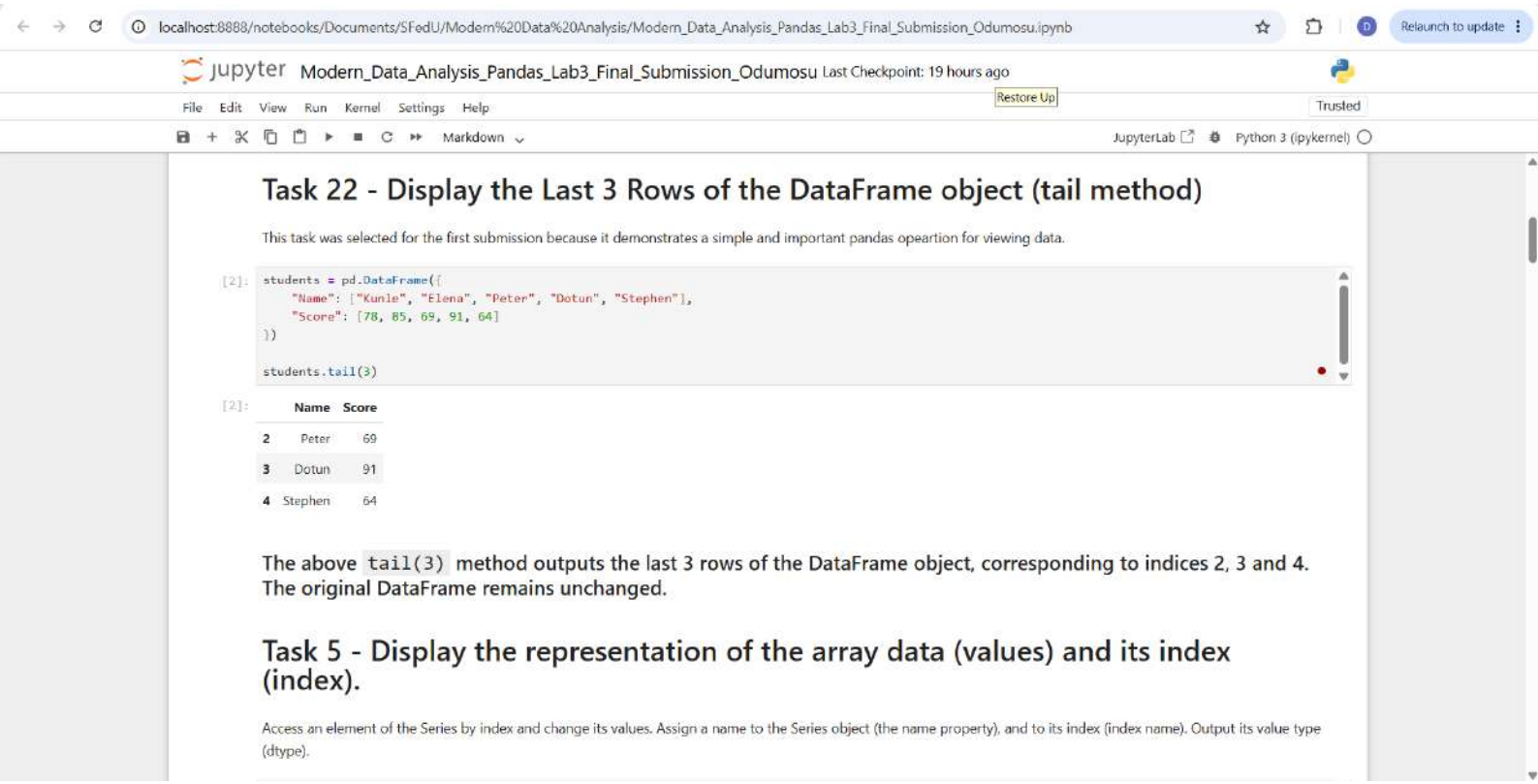

localhost:8888/notebooks/Documents/SFedU/Modern%20Data%20Analysis/Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu.ipynb
Relaunch to update
jupyter
Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu
Last Checkpoint: 19 hours ago
Restore Up
File Edit View Run Kernel Settings Help
Trusted
Markdown
JupyterLab
Python 3 (ipykernel)
Modern Data Analysis
Lab 3 - The pandas Package
Student: Oluwasomidotun Odumosu Adebowale
Student No.: 9
Assigned Tasks: 22 (previous first submission), 5, 7, 17, 35, 38, 49, 54, 56, 61, 67, and 72
Course: Modern Data Analysis
Program: MSc Computational Modeling in Technology and Finance
Lecturer: Professor Prozorov O.A.
Introduction
This notebook contains my assigned tasks from Lab 3 on the pandas package.
Pandas and NumPy Setup
I used pandas setup for data manipulation and NumPy setup to create the random matrix required by Task 7.
[1]:
import pandas as pd
import numpy as np
print("Pandas version:", pd.__version__)
print("NumPy version:", np.__version__)
Pandas version: 3.0.5
NumPy version: 2.4.6


localhost:8888/notebooks/Documents/SFedU/Modern%20Data%20Analysis/Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu.ipynb
Relaunch to update
jupyter Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu Last Checkpoint: 19 hours ago
Restore Up
File Edit View Run Kernel Settings Help
Trusted
Markdown
JupyterLab
Python 3 (ipykernel)
Task 22 - Display the Last 3 Rows of the DataFrame object (tail method)
This task was selected for the first submission because it demonstrates a simple and important pandas opeartion for viewing data.
[2]:
students = pd.DataFrame({
"Name": ["Kunle", "Elena", "Peter", "Dotun", "Stephen"],
"Score": [78, 85, 69, 91, 64]
})
students.tail(3)
[2]:
Name Score
2 Peter 69
3 Dotun 91
4 Stephen 64
The above tail(3) method outputs the last 3 rows of the DataFrame object, corresponding to indices 2, 3 and 4. The original DataFrame remains unchanged.
Task 5 - Display the representation of the array data (values) and its index (index).
Access an element of the Series by index and change its values. Assign a name to the Series object (the name property), and to its index (index name). Output its value type (dtype).

localhost:8888/notebooks/Documents/SFedU/Modern%20Data%20Analysis/Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu.ipynb

Relaunch to update

jupyter Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu Last Checkpoint: 19 hours ago

Restore Up

Trusted

File Edit View Run Kernel Settings Help

Markdown

JupyterLab Python 3 (ipykernel)

[3]:
```
scores = pd.Series(
    [72, 81, 65, 90],
    index=["A", "B", "C", "D"]
)

print("Values:")
print(scores.values)

print("\nIndex:")
print(scores.index)

print("\nValue at index B before modification:", scores["B"])
scores["B"] = 85
scores.name = "Exam Score"
scores.index.name = "Student"

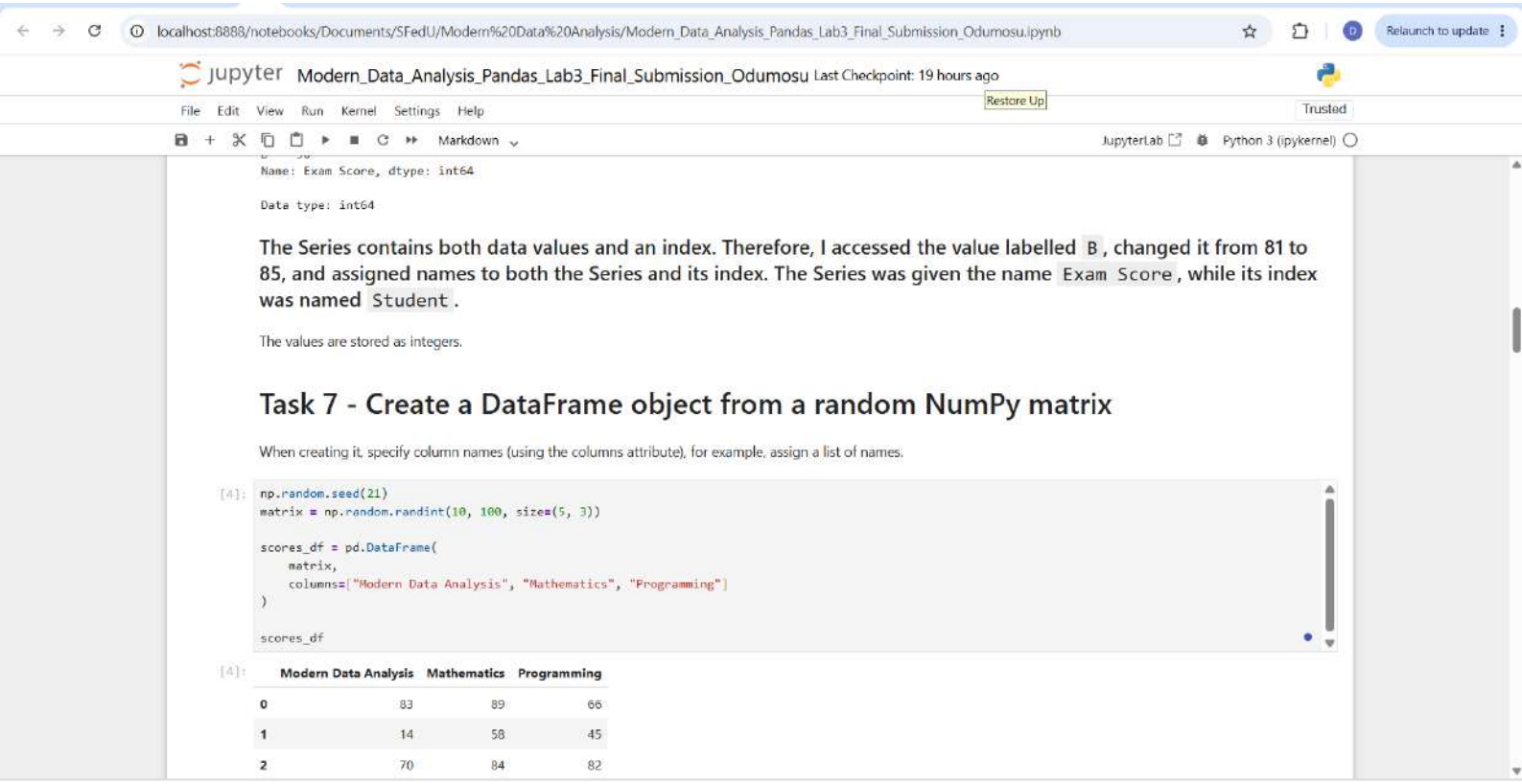
print("\nUpdated Series:")
print(scores)
print("\nData type:", scores.dtype)
```

```
Values:
[72 81 65 90]

Index:
Index(['A', 'B', 'C', 'D'], dtype='str')

Value at index B before modification: 81

Updated Series:
Student
A    72
B    85
C    65
D    90
Name: Exam Score, dtype: int64
```


localhost:8888/notebooks/Documents/SFedU/Modern%20Data%20Analysis/Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu.ipynb
Relaunch to update
jupyter Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu Last Checkpoint: 19 hours ago
Restore Up
File Edit View Run Kernel Settings Help
Trusted
Markdown
JupyterLab
Python 3 (ipykernel)
Name: Exam Score, dtype: int64
Data type: int64
The Series contains both data values and an index. Therefore, I accessed the value labelled B, changed it from 81 to 85, and assigned names to both the Series and its index. The Series was given the name Exam Score, while its index was named Student.
The values are stored as integers.
Task 7 - Create a DataFrame object from a random NumPy matrix
When creating it, specify column names (using the columns attribute), for example, assign a list of names.
[4]:
np.random.seed(21)
matrix = np.random.randint(10, 100, size=(5, 3))
scores_df = pd.DataFrame(
matrix,
columns=["Modern Data Analysis", "Mathematics", "Programming"]
)
scores_df
[4]:
Modern Data Analysis Mathematics Programming
0 83 89 66
1 14 58 45
2 70 84 82

3	73	54	71
4	58	94	69

The NumPy matrix contains five rows and three columns. The columns are named `Modern Data Analysis`, `Mathematics`, and `Programming`, making the table easier to read and use in later tasks.

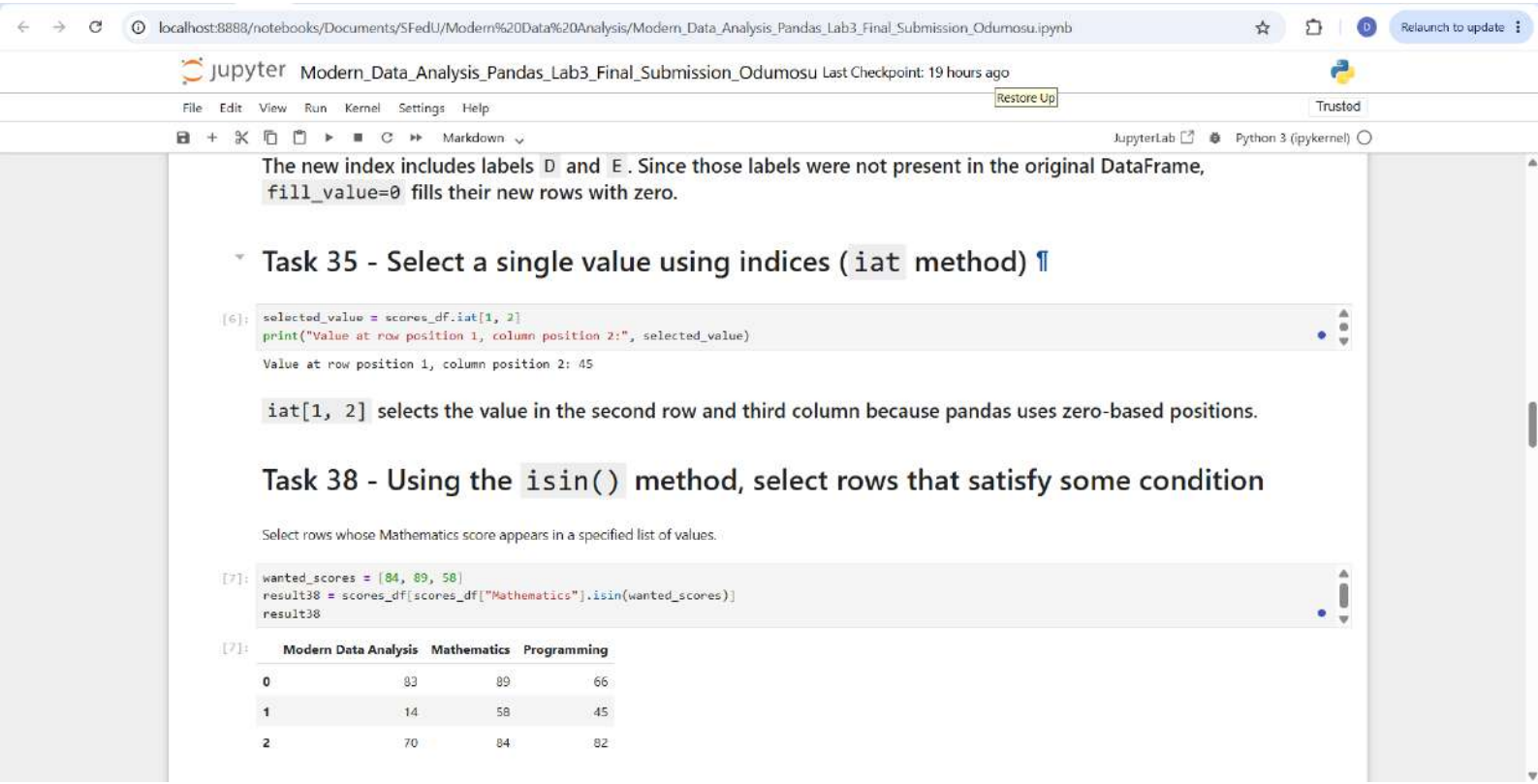
Task 17 - Change the index of an existing DataFrame table: Add New Index Elements with `reindex()` and the `fill_value` parameter

[5]:
```
data17 = pd.DataFrame(
    {"Score": [70, 82, 91]},
    index=["A", "B", "C"]
)

new_index = pd.Index(["A", "B", "C", "D", "E"])
result17 = data17.reindex(new_index, fill_value=0)
result17
```

[5]:

	Score
A	70
B	82
C	91
D	0
E	0



The new index includes labels `D` and `E`. Since those labels were not present in the original DataFrame, `fill_value=0` fills their new rows with zero.

Task 35 - Select a single value using indices (`iat` method) ¶

[6]:
```
selected_value = scores_df.iat[1, 2]
print("Value at row position 1, column position 2:", selected_value)
```

```
Value at row position 1, column position 2: 45
```

`iat[1, 2]` selects the value in the second row and third column because pandas uses zero-based positions.

Task 38 - Using the `isin()` method, select rows that satisfy some condition

Select rows whose Mathematics score appears in a specified list of values.

[7]:
```
wanted_scores = [84, 89, 58]
result38 = scores_df[scores_df["Mathematics"].isin(wanted_scores)]
result38
```

[7]:

	Modern Data Analysis	Mathematics	Programming
0	83	89	66
1	14	58	45
2	70	84	82

The `isin()` method confirmed whether each Mathematics score is contained in the requested list. Only rows with values 84, 89, or 58 are selected.

Task 49 - Select all rows in the DataFrame where the value in the first column is not missing `notna()` method

[6]:
```
data49 = scores_df.copy()
data49.loc[1, "Mathematics"] = np.nan
data49.loc[3, "Mathematics"] = np.nan

result49 = data49[data49.iloc[:, 0].notna()]
result49
```

[8]:

	Modern Data Analysis	Mathematics	Programming
0	83	89.0	66
1	14	NaN	45
2	70	84.0	82
3	73	NaN	71
4	58	94.0	69

The two Mathematics values were intentionally made missing. `data49.iloc[:, 0]` selects the first column, and `notna()` keeps rows where that column is not missing. The original `scores_df` remains unchanged because the operation uses a copy.

Task 54 - Multiply all elements in one column of the DataFrame that are less than 100 in absolute value by 10

[9]:
```
data54 = pd.DataFrame({"Value": [-120, -45, 30, 95, 140]})
condition54 = data54["Value"].abs() < 100

data54.loc[condition54, "Value"] = data54.loc[condition54, "Value"] * 10
data54
```

[9]:

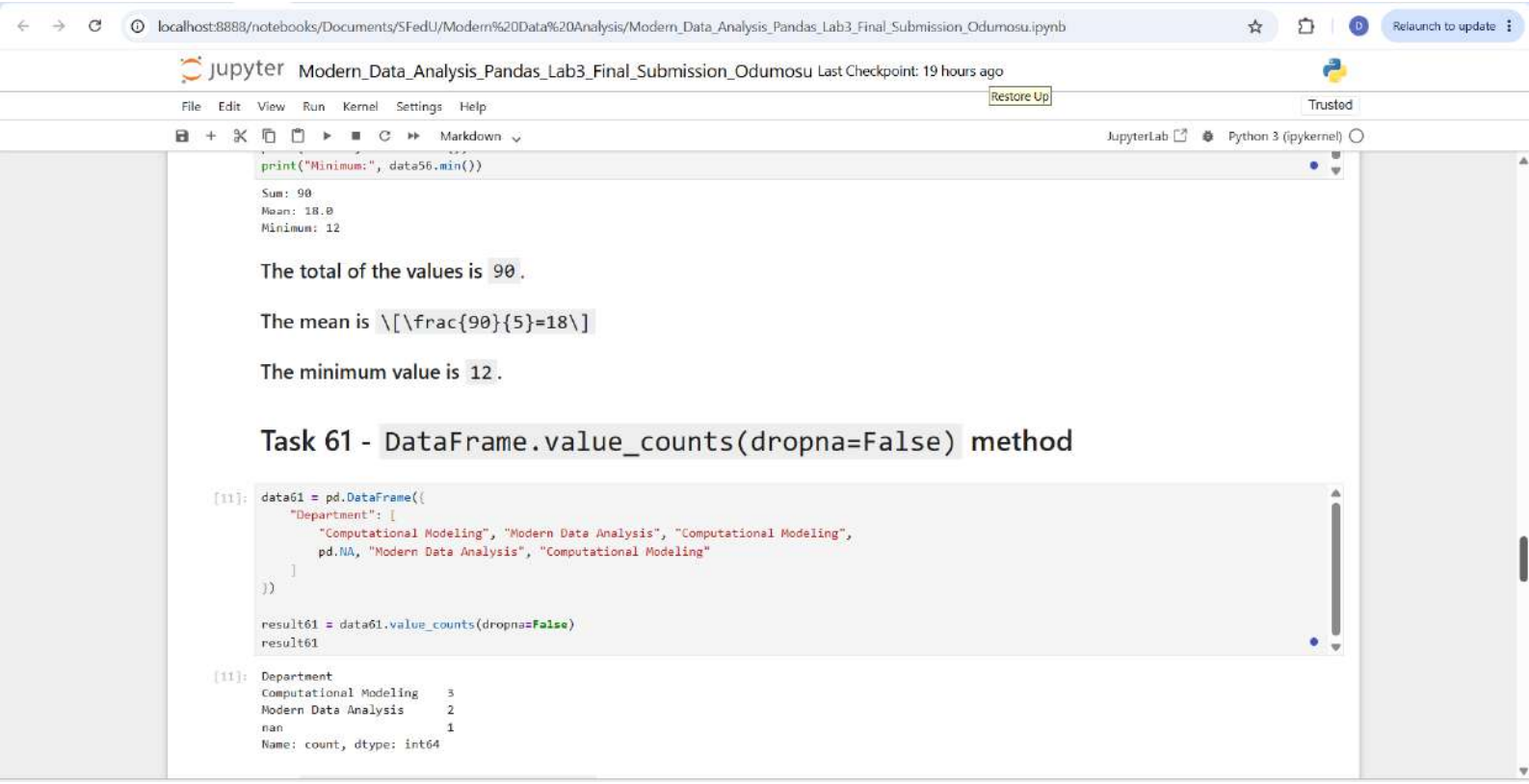
	Value
0	-120
1	-450
2	300
3	950
4	140

The condition is `\[|x| < 100\]` Therefore, the values `-45`, `30`, and `95` are multiplied by 10. The values `-120` and `140` are unchanged because their absolute values are greater than 100.

Task 56 - Sum, mean, and minimum values

[10]:
```
data56 = pd.Series([12, 18, 25, 20, 15])

print("Sum:", data56.sum())
print("Mean:", data56.mean())
```


localhost:8888/notebooks/Documents/SFedU/Modern%20Data%20Analysis/Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu.ipynb
Relaunch to update
jupyter Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu Last Checkpoint: 19 hours ago
Restore Up
File Edit View Run Kernel Settings Help
Trusted
Markdown
JupyterLab
Python 3 (ipykernel)
print("Minimum:", data56.min())
Sum: 90
Mean: 18.0
Minimum: 12
The total of the values is 90.
The mean is \[\frac{90}{5}=18\]
The minimum value is 12.
Task 61 - DataFrame.value_counts(dropna=False) method
[11]:
data61 = pd.DataFrame({
"Department": [
"Computational Modeling", "Modern Data Analysis", "Computational Modeling",
pd.NA, "Modern Data Analysis", "Computational Modeling"
]
})
result61 = data61.value_counts(dropna=False)
result61
[11]:
Department
Computational Modeling 3
Modern Data Analysis 2
nan 1
Name: count, dtype: int64

The `value_counts(dropna=False)` tells pandas not to exclude missing values from the count. Therefore, the missing value is also counted.

Task 67 - `DataFrame.where()` method

Apply the pandas `where()` method to keep values that satisfy a condition.

[12]:
```
data67 = pd.DataFrame({"Score": [45, 62, 78, 39, 91]})
result67 = data67.where(data67 >= 50)
result67
```

[12]:

	Score
0	NaN
1	62.0
2	78.0
3	NaN
4	91.0

The condition is `\[Score \geq 50\]`. Values that satisfy the condition remain unchanged. Values below 50 are replaced with `NaN`.

Task 72 - Apply min-max normalization to one column of the DataFrame

localhost:8888/notebooks/Documents/SFedU/Modern%20Data%20Analysis/Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu.ipynb

Relaunch to update

jupyter Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu Last Checkpoint: 19 hours ago

Restore Up

File Edit View Run Kernel Settings Help

Trusted

Markdown

JupyterLab Python 3 (ipykernel)

Task 72 - Apply min-max normalization to one column of the DataFrame

Transform all elements `x` of the column according to the formula: `\[x' = \frac{2(x - \min(x))}{\max(x) - \min(x)} - 1\]`, where min,max are the minimum and maximum in this column.

[13]:
```
data72 = pd.DataFrame({"Score": [10, 20, 30, 40, 50]})
minimum = data72["Score"].min()
maximum = data72["Score"].max()

if maximum == minimum:
    raise ValueError("Min-max normalization is undefined when all values are equal.")

data72["Normalized"] = (
    2 * (data72["Score"] - minimum) / (maximum - minimum) - 1
)
data72
```

[13]:

	Score	Normalized
0	10	-1.0
1	20	-0.5
2	30	0.0
3	40	0.5
4	50	1.0

The minimum value is 10 and the maximum value is 50. Therefore, the value 30 is transformed to 0.

The normalization maps is:

localhost:8888/notebooks/Documents/SFedU/Modern%20Data%20Analysis/Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu.ipynb

Relaunch to update

jupyter Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu Last Checkpoint: 19 hours ago

Restore Up

File Edit View Run Kernel Settings Help

Trusted

Markdown

JupyterLab Python 3 (ipykernel)

The normalization maps is:

the minimum value to `-1`;
the maximum value to `1`;
values between them to corresponding positions between `-1` and `1`.

Summary

The below table summarizes the assigned tasks addressed in this notebook.

[14]:
```
task_summary = pd.DataFrame({
    "Task": [22, 5, 7, 17, 35, 38, 49, 54, 56, 61, 67, 72],
    "Pandas concept": [
        "tail()", "Series properties and modification",
        "DataFrame creation with column names", "reindex() with fill_value",
        "iat", "isin", "notna", "Conditional multiplication",
        "Sum, mean, and minimum", "value_counts(dropna=False)",
        "where", "Min-max normalization to [-1, 1]"
    ],
    "Status": ["Included" for _ in range(12)]
})
task_summary
```

[14]:

	Task	Pandas concept	Status
0	22	tail()	Included
1	5	Series properties and modification	Included
2	7	DataFrame creation with column names	Included
3	17	reindex() with fill_value	Included

localhost:8888/notebooks/Documents/SFedU/Modern%20Data%20Analysis/Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu.ipynb

Relaunch to update

jupyter Modern_Data_Analysis_Pandas_Lab3_Final_Submission_Odumosu Last Checkpoint: 19 hours ago

Restore Up

Trusted

File Edit View Run Kernel Settings Help

Markdown

JupyterLab Python 3 (ipykernel)

0	22	tail()	Included
1	5	Series properties and modification	Included
2	7	DataFrame creation with column names	Included
3	17	reindex() with fill_value	Included
4	35	iat	Included
5	38	isin	Included
6	49	notna	Included
7	54	Conditional multiplication	Included
8	56	Sum, mean, and minimum	Included
9	61	value_counts(dropna=False)	Included
10	67	where	Included
11	72	Min-max normalization to [-1, 1]	Included

Conclusion ¶

This lab work gave me practical experience using pandas to work with structured data.

I practiced creating and modifying Series and DataFrames, selecting rows and values, handling missing data, calculating basic mathematics, applying conditions, counting values, and normalizing a numerical column.

These exercises strengthened my understanding of common data-analysis operations and provide a useful foundation for further work in machine learning and computational modeling.